

通过上下文折叠扩展长程 LLM 智能体

Weiwei Sun^{1,2,*}, Miao Lu^{1,3,*}, Zhan Ling¹, Kang Liu¹, Xuesong Yao¹, Yiming Yang², Jiecao Chen^{1,†}

¹ByteDance Seed, ² 卡内基梅隆大学, ³ 斯坦福大学

* 工作在 ByteDance Seed 完成, † 通讯作者

摘要

大语言模型 (LLM) 智能体在长程任务中从根本上受到上下文长度的限制。我们提出**上下文折叠** (Context Folding) 框架, 使智能体能够主动管理其工作上下文: 智能体可按过程分支到一条子轨迹以处理子任务, 在完成后将其折叠, 压缩中间步骤, 同时保留结果的简洁摘要。为使这一行为可学习, 我们开发了端到端强化学习框架 FoldGRPO, 并设计专门的过程奖励, 以鼓励有效的任务分解和上下文管理。在复杂长程任务 (深度研究和软件工程) 上, 我们的折叠智能体在保持相当或更优性能的同时, 使用的活跃上下文比 ReAct 基线小 10 $\times$, 并显著优于依赖摘要式上下文管理的模型。

日期: 2026 年 7 月 7 日

通讯作者: Weiwei Sun: sunnweiwei@gmail.com; Jiecao Chen: jiecao.chen@bytedance.com

项目主页: <https://context-folding.github.io/>

1 引言

大语言模型 (LLM) 智能体已展现出解决复杂长程问题的显著能力; 这类问题需要与环境进行广泛交互, 例如深度研究 [8, 12, 17, 21, 32] 和智能体式编码 [3, 11, 31]。有观点认为, 智能体可完成任务的长度正呈指数式增长, 其翻倍时间约为 7 个月 [20]。

然而, 将 LLM 智能体扩展到更长时间跨度, 根本上受限于智能体框架的设计 [35]。这类框架将完整交互历史 (推理、工具调用和观察) 线性累积到一个持续扩张的上下文中。随着时间跨度增长, 由此产生长上下文挑战: (1) 性能下降, 因为 LLM 难以在极长上下文中利用相关信息 [14, 18, 28]; (2) 效率低下, 源于注意力机制的二次复杂度, 以及管理 KV-cache 日益增加的开销 [13]。

现有扩展长程 LLM 智能体的方法大致分为两类: (1) 基于摘要的方法: 当工作上下文已满时触发事后摘要阶段 [1, 19, 24, 34, 38, 43]。尽管这会压缩上下文, 却可能突然打断智能体的工作上下文和推理流, 导致次优结果。(2) 多智能体系统: 将任务分配给专门智能体以管理上下文长度 [2, 33, 40, 41]。但这些

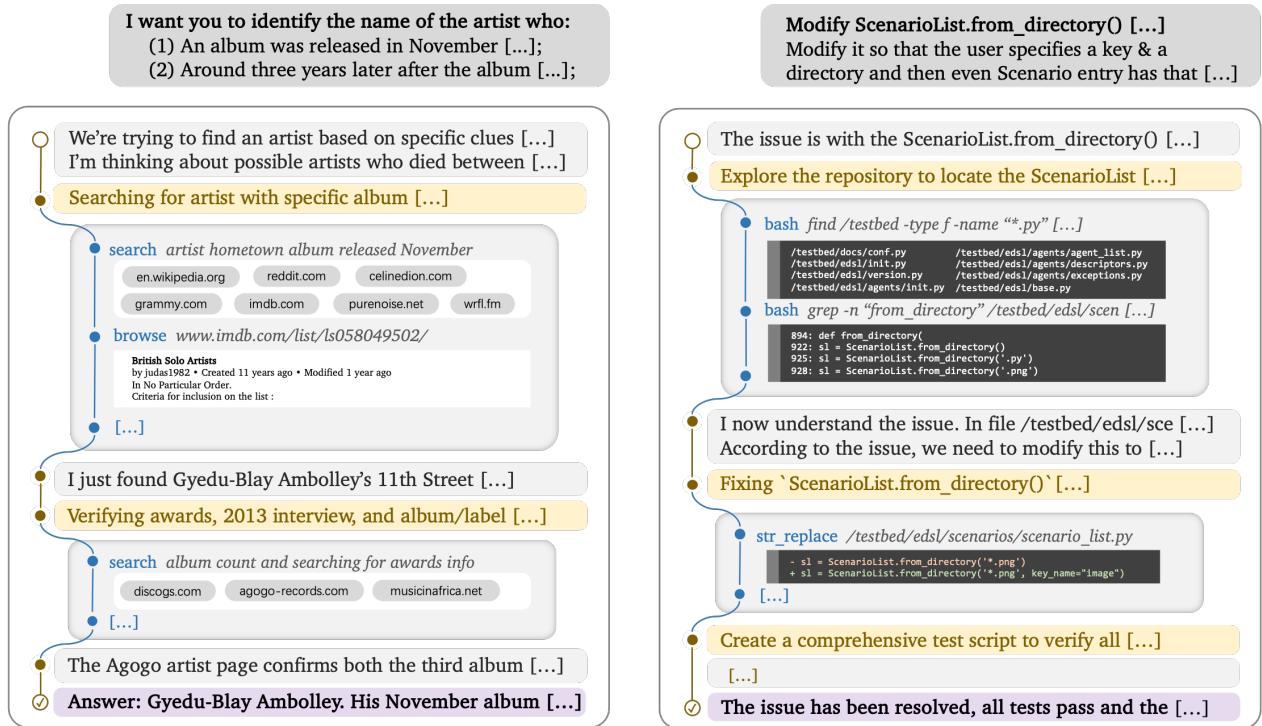


Figure 1 长程任务中的上下文折叠示例：深度研究（左）和智能体式编码（右）。

系统通常依赖手工设计、面向特定问题的工作流，难以泛化，也难以端到端优化。

本文提出**上下文折叠**（Context Folding）：一种使模型能够主动管理工作上下文的智能体机制。具体而言，智能体使用两种特殊动作管理上下文：(i) **branch** 动作，为局部子任务创建临时子轨迹；(ii) **return** 动作，总结结果并重新并入主线程。此后，分支中的中间步骤被“折叠”——即从上下文中移除——只保留 **return** 调用中的简洁摘要。图 1 展示了该过程在深度研究和智能体式编码任务中的应用：智能体将词元密集型操作（如网页搜索或代码库探索）卸载至分支，只为高层推理保留关键发现与洞见。与现有方法相比，上下文折叠实现了主动上下文管理：智能体的短期上下文不被打断，长期上下文则自动管理。

基于上下文折叠框架，我们提出一种新颖的端到端强化学习算法，用于在复杂长程任务上训练 LLM 智能体。核心创新是 **FoldGRPO**：它在标准 GRPO 的基础上加入 (i) 动态折叠的 LLM 上下文；(ii) 直接引导上下文折叠行为的稠密、词元级过程奖励。具体而言，RL 算法通过未折叠词元惩罚，教导模型将问题有效分解为可在分支中处理的局部子任务，避免在主上下文中执行词元密集型操作。此外，模型通过越界惩罚学习在子任务内保持聚焦，并在摘要中保留有助于最终目标的关键信息。掌握这些技能后，智能体能够处理远得多的交互历史，使框架扩展智能体的有效时间跨度并提升整体系统效率。

我们在两个长程基准 BrowseComp-Plus [6] 和 SWE-Bench Verified [11] 上评估方法，智能体以显著效率取得强劲性能。尽管使用由最多 10 个分支管理的紧凑 32K 活跃词元预算，我们的**折叠智能体**在两个基准上分别获得 62.0% 和 58.0% 的 pass@1，超过需要巨大 327K 上下文窗口的基线，并显著优于基于上下文摘要的方法。强化学习是方法有效性的根源：在 BrowseComp-Plus 上带来 20.0% 的绝对提升，在 SWE-Bench 上带来 8.8% 的绝对提升。进一步分析表明，智能体学会调用更多工具并生成更长输出以处理复杂问题，并能在推理时扩展至更大词元预算来应对更具挑战性的任务。总体而言，这些结果表

明，学习主动管理上下文，而非仅仅扩展或启发式压缩上下文，是通向可扩展长程智能体的一条有原则路径。

总之，本文有三项贡献：

- (i) 提出**上下文折叠**机制，使智能体能够主动管理上下文，并缓解历史线性增长的挑战。
- (ii) 提出 **FoldGRPO** 强化学习框架，结合动态折叠 LLM 上下文和稠密过程奖励，训练智能体有效获得上下文折叠能力。
- (iii) 在长程基准上展示有前景的性能，说明该方法为构建更强 LLM 智能体提供了一条可扩展、可延展的路径。

1.0.1 过程奖励设计

在 RLVR 中，通常依据任务成功或失败，以标准二值结果奖励优化智能体。但我们实证发现，这种稀疏奖励信号不足以学习有效的上下文折叠。具体而言，出现两种关键失败模式：(i) 智能体无法进行策略性规划，将词元密集型操作留在主上下文中未折叠，迅速耗尽可用词元预算；(ii) 智能体难以妥善管理分支：子任务完成后往往没有从子分支返回，而是在同一分支中继续之后的工作。为有效优化折叠智能体，我们分别为主轨迹词元和分支轨迹词元引入词元级过程奖励。

未折叠词元惩罚。主线程总上下文长度超过工作上下文上限的 50% 时，我们对主线程内的全部词元施加 $Q_{i,t} = -1$ ，创建分支的轮次中的词元除外。这会惩罚智能体在主线程中执行词元密集型操作，并鼓励智能体在独立分支中执行这些操作。

越界惩罚。对于每个分支，我们使用 GPT-5-nano 根据分支提示和返回消息判断智能体是否进行了指定子任务之外的操作。若有，则对此分支内全部词元施加 $Q_{i,t} = -0.2$ ，惩罚此类越界行为。这会鼓励智能体仅执行当前分支所给出的精确子任务。

失败惩罚。我们对失败工具调用轮次中的全部词元施加 $Q_{i,t} = -1$ 。其他情形下， $Q_{i,t} = 0$ 。

1.1 上下文折叠与其他方法的关系

与多智能体系统的关系。上下文折叠可解释为一般多智能体系统的一种特定形式：主智能体将子任务委派给子智能体。相较流行多智能体系统 [9]，我们的设计有如下不同：(i) 上下文折叠不采用预定义子智能体，子智能体由主智能体按需创建；(ii) 所有智能体共享相同的上下文前缀，因此对 KV-cache 友好；(iii) 主智能体与子智能体交替运行，而非并行运行。

与基于上下文摘要方法的关系。相较启发式的摘要式上下文管理 [21, 38]，后者在任意位置丢弃细节，上下文折叠可视为与子任务边界对齐的可学习摘要机制。这确保推理在执行期间被保留，只有在其效用实现后才被压缩。

2 实验

2.1 数据集

我们在两个代表性长程智能体任务上开展实验：深度研究和智能体式软件工程。

深度研究。我们使用 BrowseComp-Plus (BC-Plus) [6], 它在原始 BrowseComp 数据的基础上补充了经过验证的语料库。检索器使用 Qwen3-Embed-8B。鉴于训练数据质量对 BrowseComp 任务至关重要, 而现有数据集通常未开源 [15, 24], 我们将 BrowseComp-Plus 划分为训练集与评估集, 以解耦数据分布的影响。划分后有 680 个训练样本和 150 个评估样本。深度研究任务使用 `search(query, topk)` 与 `open_page(url)` 工具, 奖励由官方基于 LLM 的评审器给出 [6]。

智能体式软件工程。我们使用 SWE-Bench Verified (SWEB-V) [11] 作为评估集。为收集训练数据, 我们在开源数据集 SWE-Gym [23] 和 SWE-Rebench [4] 的子集上, 对基线智能体¹进行八次 rollout, 保留成功率介于 0 和 87.5% 之间的样本, 最终得到 740 个样本。SWE 任务使用 `execute_bash`、`str_replace_editor` 和 `think` [31] 工具, 奖励通过在样本特定的沙箱环境中运行单元测试获得。

对于两项任务, 我们将测试样本划为简单、中等和困难三个难度等级。BrowseComp-Plus 的分类方法是对每个样本运行 ReAct 智能体 8 次: acc@8 分数 $\geq 87.5\%$ 标为简单, 分数为 0% 标为困难, 其余标为中等, 每个等级各有 50 个样本。SWE-Bench Verified 的分类依据原始数据集的 time-to-resolve 指标: 简单 (≤ 15 分钟, 194 个样本)、中等 (15 分钟至 1 小时, 261 个样本) 和困难 (≥ 1 小时, 45 个样本)。

各数据集的系统提示细节见附录 B。

2.2 实现

我们使用 Seed-OSS-36B-Instruct²作为基础 LLM, 并在其上进行 RL 训练。RL 训练基于 VeRL, rollout batch size 设为 32, 组大小设为 8, PPO batch size 设为 128, 学习率为 1×10^{-6} , 不使用 KL 项, clip high 为 0.28, clip low 为 0.2。我们采用异步 rollout, 最大 off-policy step 为 5。训练期间, 我们通过为每个分支构建独立、因果条件化的上下文来实现上下文折叠操作 $\mathcal{F}$, 以提高训练效率 (详见附录 A)。模型训练 50 步 (约 2 个 epoch)。对于折叠智能体, LLM 最大上下文长度设为 32,768, 最多允许 10 个分支, 理论最大词元数为 327,680。推理时采用贪婪解码 (即 temperature = 0)。

2.3 基线

我们与以下基线比较:

ReAct 智能体 [36], 它保留全部上下文。我们考虑不同上下文长度: (a) 短上下文: 32,768 个词元, 与本方法上下文长度相同; (b) 中等上下文: 中间长度, 例如 65,536 和 131,072; (c) 长上下文: 327,680 个词元, 与最大总词元成本相同。

摘要智能体 [34, 38], 当上下文已满时调用摘要。为公平比较, 我们将最大上下文长度设为 32,768, 并允许 10 次摘要会话。

对于两个直接可比基线, 我们均使用相同的基础模型 (Seed-OSS-36B-Instruct)、数据、基础设施和 RL 训练超参数。除此之外, 还在两项任务上将方法与先前的闭源和开源系统比较, 包括 GPT-5、GPT-4.1、DeepSeek-V3.1 (2509)、GLM-4.5-Air 和 Qwen3-235B-A22B-Instruct-2507。

3 实验结果

¹使用 OpenHands 的 Seed-OSS-36B-Instruct, 响应长度为 65,536。

²<https://huggingface.co/ByteDance-Seed/Seed-OSS-36B-Instruct>

模型	峰值长度	最大词元	BrowseComp-Plus		SWE-Bench Verified	
			Pass@1	工具调用	Pass@1	工具调用
使用 100B+ LLM 的 ReAct 智能体						
GPT-5	327K	327K	0.793	14.2	0.718	42.6
GPT-4.1	327K	327K	0.640	5.6	0.486	28.7
DeepSeek-V3.1	327K	327K	0.613	10.6	0.610	53.2
GLM-4.5-Air	327K	327K	0.566	11.1	0.576	51.2
Qwen3-235B-A22B	327K	327K	0.560	12.8	0.344	32.1
ReAct 智能体						
Seed-OSS-36B	32K	32K	0.286 (-19.2)	3.8	0.436 (-11.6)	25.8
+ RL (GRPO)	32K	32K	0.446 (-3.2)	5.5	0.480 (-7.2)	27.8
Seed-OSS-36B ^ψ	327K	327K	0.478 (+0.0)	10.8	0.552 (+0.0)	49.5
+ RL (GRPO)	327K	327K	0.540 (+6.2)	10.2	0.574 (+2.2)	55.4
摘要智能体						
Seed-OSS-36B	32K	32K × 10	0.386 (-9.2)	17.4	0.488 (-6.4)	77.0
+ RL (GRPO)	32K	32K × 10	0.527 (+4.9)	18.0	0.550 (-0.2)	74.9
折叠智能体 (本文)						
Seed-OSS-36B	32K	32K × 10	0.420 (-5.8)	12.9	0.492 (-6.0)	72.8
+ RL (GRPO)	32K	32K × 10	0.567 (+8.9)	16.0	0.564 (+1.2)	79.5
+ RL (FoldGRPO)	32K	32K × 10	0.620 (+14.2)	19.2	0.580 (+2.8)	96.5

Table 1 BrowseComp-Plus (N=150) 和 SWE-Bench Verified (N=500) 上的性能。粗体表示表现最佳的 36B 模型。括号中的数字表示相对于 327K ReAct 智能体 Seed-OSS-36B 基线^ψ的提升或降低。

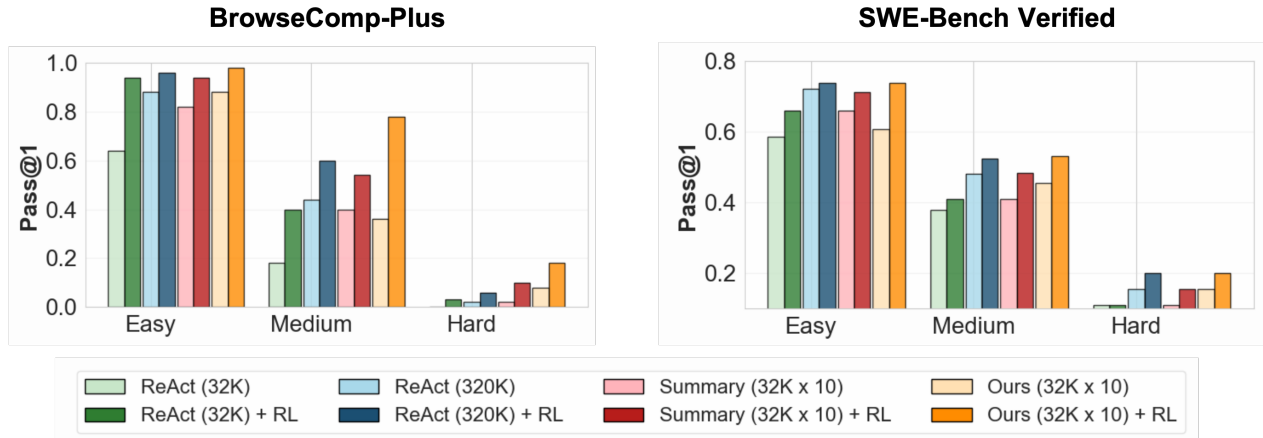


Figure 2 不同数据难度组上的智能体性能。RL 训练在简单、中等和困难样本上均带来一致性能增益。

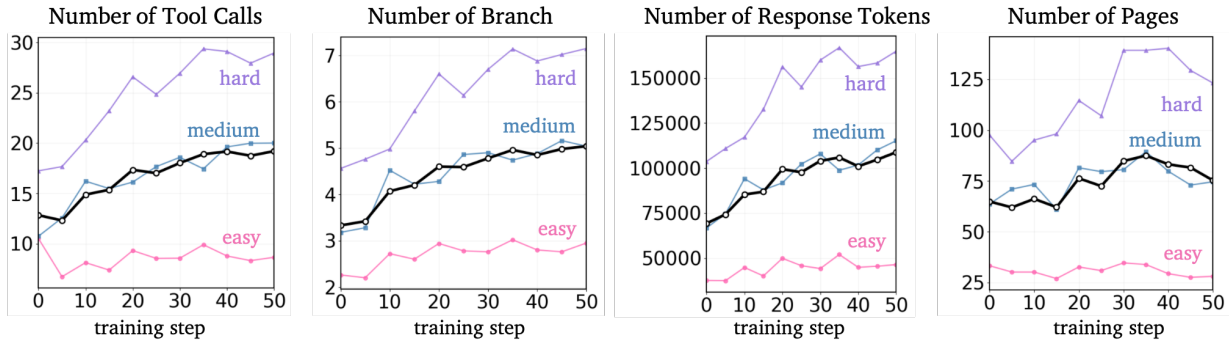


Figure 3 经过 RL 训练后，工具调用次数、分支行为、总词元数和已搜索页面数均有所增加。

3.1 主要结果

表 1 汇总了 BrowseComp-Plus 和 SWE-Bench Verified 上的主要结果。结果凸显强化学习对于释放上下文折叠能力的关键作用。

	BrowseComp-Plus				SWE-Bench Verified			
	完成	主长度	范围	分支数	完成	主长度	范围	分支数
折叠智能体 (Seed-OSS-36B)	0.806	12,195	0.774	3.51	0.781	47,475	0.473	3.05
+ RL (GRPO)	0.738	22,285	0.762	3.88	0.612	48,908	0.419	3.80
+ RL (FoldGRPO)	0.935	7,752	0.895	4.98	0.962	8,885	0.754	5.90

Table 2 不同优化方法的模型行为统计。 FoldGRPO 鼓励聚焦分支和压缩后的主上下文，同时提高任务范围准确率与完成率。

起初，在未进行 RL 时，上下文折叠智能体已超过 32K 上下文 ReAct 和上下文摘要基线，但尚未匹配长上下文 ReAct 智能体的性能。经过 RL 训练，智能体性能显著提升：BrowseComp-Plus 的 *pass@1* 达到 **0.620 (+20%)**，SWE-Bench Verified 达到 **0.580 (+8.8%)**。它不仅超过包括最大长度同为 327K 的长上下文 ReAct 智能体在内的所有基线，还取得与基于大得多的 100B+ 参数模型构建的智能体相当的性能。

进一步分析得到两项关键洞见。第一，消融研究确认所提出的 **FoldGRPO 至关重要**：比基础 GRPO 算法显著更优（例如 BrowseComp 上 +7.7%，SWE-Bench 上 +1.6%）。第二，性能增益与工具调用频率增加相关，而 RL 训练会进一步鼓励工具调用。这表明框架使智能体能够更彻底地探索环境，从而发现更稳健的解决方案。

3.2 按任务难度划分的性能

图 2 按任务难度拆解了智能体性能，比较强化学习前后的得分。结果清楚表明，RL 训练在简单、中等和困难样本上均带来一致的性能提升；尤其是中等和困难子集上的改善明显更大。这一发现凸显了智能体处理复杂问题的增强能力，而这些问题需要更精细的长上下文管理。

图 3 展示了 BrowseComp-Plus 上 RL 训练期间智能体的学习动态。随着训练推进，智能体的工具调用、分支创建、响应词元数和搜索页面数均稳步增加；这种增长在更困难样本上最为显著。例如，困难子集的响应长度从约 100K 上升至超过 160K 词元。这些结果说明智能体学会为复杂问题分配更多交互与计算，采用更具适应性且更有效的问题求解策略。

3.3 RL 算法消融

为理解所提出的 FoldGRPO 如何塑造智能体行为，我们分析表 2 中的关键统计量。这些指标包括上下文限制内的任务完成率 (Finish)、主轨迹长度 (Main Len)、子轨迹保持主题相关的准确率 (Scope) 和创建的分支数量 (# Branch)。可看到，使用标准 GRPO 基线训练会产生较差行为：智能体有较低的 Finish 率，生成过长轨迹，并在子任务中失去聚焦，体现为 Scope 准确率下降。这表明其未能有效管理上下文。

相反，FoldGRPO 修正了这些问题。它鼓励聚焦分支，显著提高 Scope 准确率和 Finish 率。最值得注意的是，在处理超过 100K 总词元的同时，它将主轨迹压缩到约 8K 词元，实现超过 **90% 的上下文压缩**，展示了智能体将长交互压缩为紧凑且有用历史的能力。

3.4 按上下文长度划分的性能

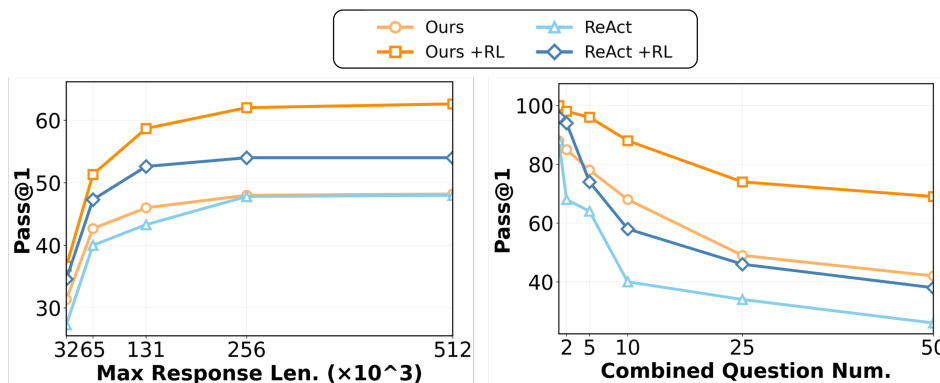


Figure 4 左: Pass@1 与智能体最大上下文长度的关系。右: Pass@1 与组合问题数量的关系。将多个简单问题合并为一个更难的问题以提高问题复杂度; 组合问题越多, 正确回答所需的动作越多、上下文越长。细节见第 3.4.2 节。

3.4.1 上下文长度的影响

为考察性能如何随上下文长度扩展, 我们在 BrowseComp 上评估方法, 并将分支数从 0 变化到 16。如图 4 (左) 所示, 方法始终超过 ReAct, 强化学习带来进一步增益。然而, 超过 320K 词元后性能趋于平台, 因为多数任务样本已被完成, 额外上下文的收益有限。

3.4.2 任务复杂度的影响

遵循 Zhou et al. [43], 我们将多个问题合并为智能体必须在一次会话中回答的复合查询, 以提高任务复杂度 (示例见图 5)。图 4 (右) 展示 1 至 50 个组合问题时的结果。该设置下, 我们允许无限制分支, 并将 ReAct 的上下文限制设为 1M 词元。随着任务复杂度增加, 上下文折叠的优势更为明显, 展示了强大的长度泛化能力。值得注意的是, 尽管智能体训练时的任务最多需要 10 个分支, 它在解决包含 50 个问题的任务时, 能自适应地平均使用 32.6 个分支。

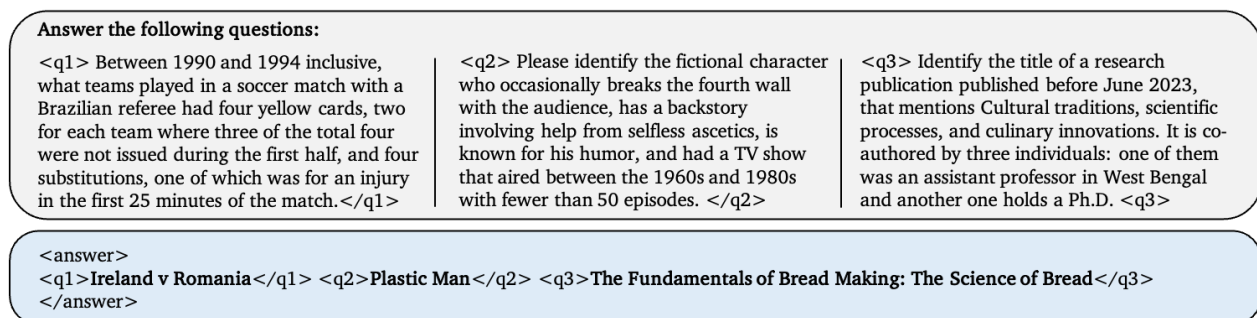


Figure 5 第 3.4.2 节所述组合问题实验中模型输入和输出的示例。本例将 3 个简单问题合并为一个更难的问题。

3.5 进一步分析

3.5.1 案例研究

图 6 展示了上下文折叠智能体在 BrowseComp-Plus 上的定性案例。给定一个需要寻找满足特定条件的研究出版物的查询, 智能体先探索高层主题并识别候选项; 随后搜索以验证条件, 获得关键洞见但未确认所有要求; 接着扩大搜索范围, 找到正确答案。在此过程中, 4 个分支将完整的 107K 词元上下文压

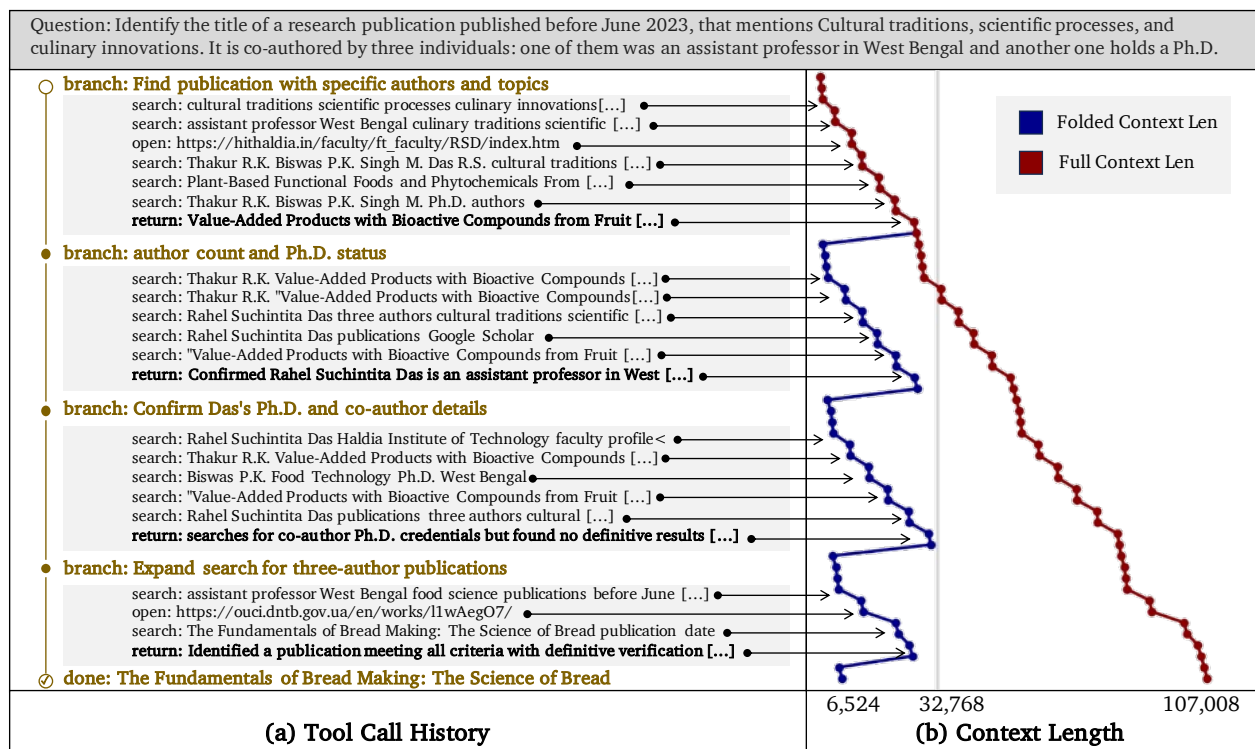


Figure 6 BrowseComp-Plus 上智能体工具调用历史和上下文长度的示例。

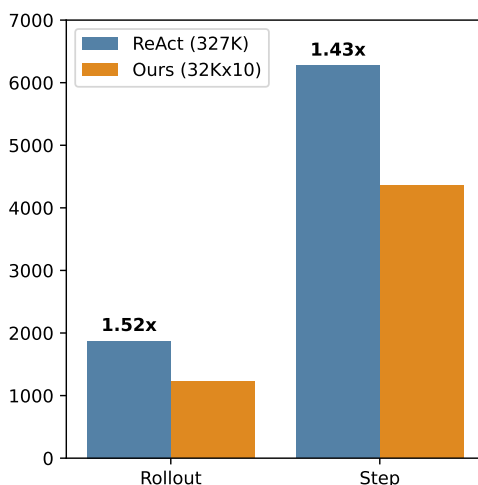


Figure 7 训练时间成本。图中展示 rollout 和每个训练步骤的逐步平均时间。

缩到仅 6K。

3.5.2 训练速度

图 7 展示 rollout 和每个训练步骤的逐步平均时间。我们观察到，327K ReAct 模型每步需要更长训练时间。注意，我们采用异步 rollout（附录 A.2）；这里的 rollout 时间仅度量主线程在 rollout 上的时间成本。

3.5.3 并行分支

折叠智能体能否从并行分支中获益仍是开放问题；这里的并行分支是指同时创建多个并行运行的子分支。我们在 BrowseComp-Plus 上训练了一个使用并行分支的智能体，其他设置与单分支智能体相同。并行分支版本在 BrowseComp-Plus 上达到 0.6133 Pass@1，超过基线但与单分支版本表现相近。此外，训练后，并行分支智能体平均创建约 2.3 个并行分支，并阅读更多网页（110 对单分支版本的约 80）。但它未取得更高分数，可能是因为任务特征更偏向深度优先。对具有广度优先结构的其他任务（例如 WideSearch [33]）而言，研究 LLM 智能体的并行性可能更有希望。

4 相关工作

LLM 智能体的快速演进，源于对复杂长程任务中更高自主性的追求 [11, 16, 20, 22, 42]。建立在集成规划、记忆和工具使用的智能体架构之上 [30]，研究已从简单顺序推理发展到用于探索和问题求解的动态多路径策略 [5, 10, 26, 37]。但这种进展也揭示了一项关键瓶颈：智能体工作上下文是有限且昂贵的 [1, 35]。

上下文管理策略主要有两种范式：上下文摘要，即智能体从外部记忆存储中卸载和检索信息 [27, 29, 34, 38, 43]；以及多智能体协作，即在具备聚焦上下文的专门智能体之间划分任务 [2, 33, 40, 41]。两种范式都将上下文管理表述为架构或检索问题，而缺少一种集成方法：让它成为可学习的认知技能，而不是外部功能。

强化学习（RL）可通过环境或人类反馈有效地锚定智能体 [24, 39]，但主要关注外在任务成功 [7]。如何训练智能体管理自身工作记忆等内在技能，仍是探索不足的研究方向。本文通过将上下文管理表述为可学习技能，并使用过程级奖励直接教授该技能，为这一新兴前沿作出贡献。

5 结论与未来工作

本文提出**上下文折叠**：一种通过选择性折叠短暂子轨迹、仅保留与决策相关的必要信息来管理长程轨迹的智能体机制。结合强化学习框架，上下文折叠能够在树状轨迹间进行高效信用分配，并在长程编码和深度研究任务中取得显著改善。两项长上下文任务上的实证结果表明，折叠使智能体能匹配或超过具有更大上下文窗口的基线性能，同时相较基于摘要的压缩提高效率 and 稳定性。一个有前景的未来方向是多层上下文折叠，即发展层次化折叠策略，使折叠本身可进一步折叠，从而获得更深度压缩。

致谢

作者感谢 Weihua Du、Guanghao Ye、Joey Hong、Bowen Xiao、Ting-Han Fan 和 Lingfeng Shen 在本工作准备期间提供的宝贵讨论与反馈。

6 方法

6.1 基础建模

给定问题 q ，智能体生成一条多轮交互轨迹：

$$\tau := (a_1, o_1, a_2, o_2, \dots, a_T, o_T),$$

这里， $\mathbf{1}_{\tau_{i,t}}^{\text{LLM}}$ 确保只优化 LLM 生成的词元，并掩蔽工具观察产生的词元。下文解释以红色标示的 Fold-GRPO 两项关键特征。

- (i) **上下文折叠**。不同于基础多轮 LLM RL 算法在优化策略时将完整交互历史追加到上下文，FoldGRPO 将上下文管理器 $\mathcal{F}(\cdot)$ 应用于历史 $\tau_{i,<t}$ ，根据分支-返回动作折叠用于词元 $\tau_{i,t}$ 的上下文。
- (ii) **过程奖励信号**。在计算优势 $\hat{A}_{i,t}$ 时，加入词元级过程奖励 $Q_{i,t}$ 来正则化模型的分支-返回行为，详细说明见下一节。

其中 a_i 是第 i 步的 LLM 输出（包括推理和工具调用）， o_i 是对应的工具调用结果。基础 ReAct 风格智能体 [35] 将交互建模为：

$$p_{\theta}^{\text{ReAct}}(\tau | q) = \prod_{i \in [T]} \pi_{\theta}(a_i | q, (a_1, o_1, \dots, a_{i-1}, o_{i-1})),$$

即每次生成 LLM 输出时都将完整交互历史追加到上下文。可是，在深度研究和智能体式编码等长程任务中， τ 会因广泛交互快速积累，长到超出工作上下文限制。上下文扩张时，模型的推理和指令遵循能力也可能下降，给完成长程任务带来进一步挑战。

6.2 我们的方法：上下文折叠

为应对上述挑战，我们引入上下文折叠，使智能体能够通过分支与折叠主动管理工作上下文。具体而言，我们设计了供智能体调用的两种上下文管理工具。从解决问题 q 的主线程开始，智能体可以：

- (i) **branch**(description, prompt)：从主线程分支出一个独立工作上下文，以完成为解决 q 所需的子任务 q' 。其中 description 是对子任务的简短概述，prompt 是该分支的详细指令。该工具返回一条表明分支已创建的模板消息。
- (ii) **return**(message)：折叠在该分支中生成的上下文并返回主线程。其中 message 描述此分支的结果。调用该工具后，智能体上下文切回主线程，并追加来自分支的模板化 message。

借助这两种工具，智能体可通过 (i) 分支出独立工作上下文来解决独立子任务；(ii) 折叠分支中的中间步骤，仅将分支结果追加到主线程以恢复执行，从而主动管理上下文。形式化地，上下文折叠智能体建模为：

$$p_{\theta}^{\text{Context Fold}}(\tau | q) := \prod_{i \in [T]} \pi_{\theta}(a_i | q, \mathcal{F}(\tau_{<i})). \quad (1)$$

其中 $\tau_{<i} = (a_1, o_1, \dots, a_{i-1}, o_{i-1})$ 表示步骤 i 前所有动作-观察对的完整历史， $\mathcal{F}$ 是上下文管理器，它折叠 **branch** 与 **return** 工具调用之间的交互历史。下例中，上下文管理器折叠之前分支内的全部动作-观察对：

$$\begin{aligned} \mathcal{F}(a_1, o_1, \textcolor{red}{a_2}, \underbrace{\textcolor{red}{o_2}, a_3, o_3, \textcolor{blue}{a_4}}_{\text{分支 1}}, \textcolor{blue}{o_4}, \textcolor{red}{a_5}, \underbrace{\textcolor{red}{o_5}, a_6, o_6, a_7, o_7, \textcolor{blue}{a_8}}_{\text{分支 2}}, \textcolor{blue}{o_8}, a_9, o_9, \textcolor{red}{a_{10}}, o_{10}) \\ \rightarrow (a_1, o_1, \textcolor{red}{a_2}, \textcolor{blue}{o_4}, \textcolor{red}{a_5}, \textcolor{blue}{o_8}, a_9, o_9, \textcolor{red}{a_{10}}, o_{10}), \end{aligned}$$

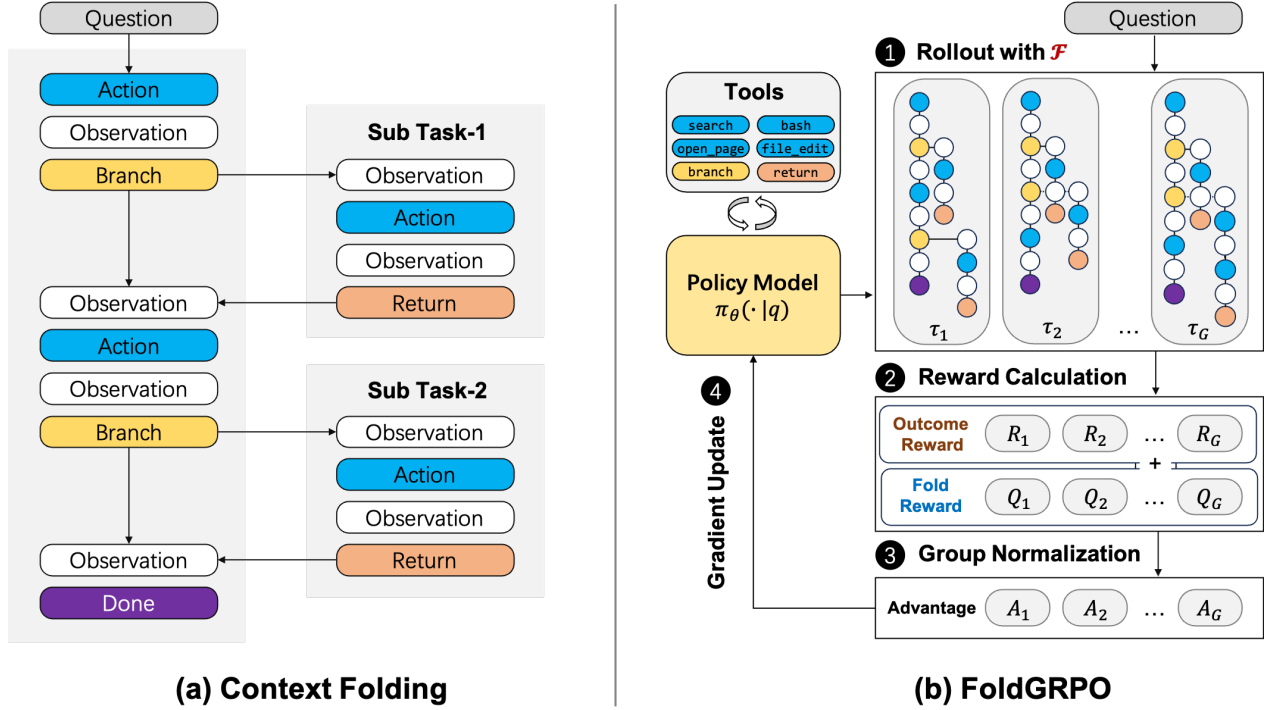


Figure 8 (a) 上下文折叠：通过分支和返回使智能体能够主动管理上下文的机制。(b) FoldGRPO：上下文折叠智能体的端到端优化。

因此， a_2 与 a_4 之间，以及 a_5 与 a_8 之间的片段被折叠。

推理效率。推理期间，智能体管理一个上下文 KV-cache：调用 `return` 动作时，它会将 KV-cache 回滚到对应的 `branch` 位置，因为该位置的上下文前缀与调用 `branch` 动作前相同。这使上下文折叠在推理上具有效率。

实例化：规划-执行。为将上下文折叠实例化到长程任务中，我们采用规划-执行框架，智能体在两种状态间交替：(i) 规划状态：智能体在主线程中执行高层推理、分解任务，并决定何时为子任务创建分支。在该状态中，不鼓励词元密集型工具使用，使主上下文专注于高层策略。(ii) 执行状态：智能体在活跃分支中完成已分配的子任务。为保持清晰结构并防止嵌套复杂性，执行状态中禁用创建新分支。

6.3 FoldGRPO：上下文折叠智能体的端到端 RL

为优化上下文折叠智能体，本节引入端到端 RL 训练框架，即 Folded-context Group Relative Policy Optimization (FoldGRPO)。FoldGRPO 联合优化完整交互轨迹，包括主线程和子任务分支；同时依据上下文折叠模型 (1) 折叠 rollout 历史，从而在训练时维持紧凑的工作上下文。此外，FoldGRPO 采用新颖的过程奖励设计，高效引导智能体分支行为的训练。我们先在第 6.3.1 节介绍整体算法设计，再在第 1.0.1 节给出过程奖励设计。

6.3.1 整体算法设计

在 FoldGRPO 的每个训练步骤中，对训练数据集 $\mathcal{D}$ 中的任务 q ，根据上下文折叠模型 (1) 从旧策略 π_{old} 采样 G 条轨迹 $(\tau_1, \tau_2, \dots, \tau_G)$ 。每条完整轨迹，例如 $\tau_i = (a_{i,1}, o_{i,1}, \dots, a_{i,T}, o_{i,T})$ ，均可视为由词元构

成的序列: $\tau_i = [\tau_{i,1}, \dots, \tau_{i,|\tau_i|}]$ 。遵循基于可验证奖励的强化学习 (RLVR) 配方, 每条轨迹 τ_i 拥有最终奖励 $R_i \in \{0, 1\}$ 。

学习目标。 FoldGRPO 的学习目标定义为:

$$\mathcal{J}_{\text{FoldGRPO}} = \mathbb{E}_{\substack{q \sim \mathcal{D}, \\ \{\tau_i\}_{i=1}^G \sim \pi_{\text{old}}(\cdot|q)}} \left[\frac{1}{\sum_{i=1}^G |\tau_i|} \sum_{i=1}^G \sum_{t=1}^{|\tau_i|} \min \left\{ r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}_{i,t} \right\} \right],$$

其中, 重要性采样比率和组相对优势估计器 [25] 为:

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(\tau_{i,t} | q, \mathcal{F}(\tau_{i,<t}))}{\pi_{\theta_{\text{old}}}(\tau_{i,t} | q, \mathcal{F}(\tau_{i,<t}))} \cdot \mathbf{1}_{\tau_{i,t}}^{\text{LLM}}, \quad \hat{A}_{i,t} = \frac{\text{clip}(R_i + Q_{i,t}, 0, 1) - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}.$$

References

- [1] All-Hands.dev. Openhands: Context condensation for more efficient ai agents, April 2025.
- [2] Anthropic. How we built our multi-agent research system, June 2025.
- [3] Anthropic. Claude code, 2025.
- [4] Ibragim Badrtdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *ArXiv*, abs/2505.20411, 2025.
- [5] Maciej Besta, Nils Blach, Ale Kubek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoeffler. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2023.
- [6] Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhui Chen, and Jimmy Lin. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. *ArXiv*, abs/2508.06600, 2025.
- [7] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Jun-Mei Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiaoling Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bing-Li Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dong-Li Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Jiong Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, M. Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, Ruiqi Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shao-Kang Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wen-Xia Yu, Wentao Zhang, Wangding Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyu Jin, Xi-Cheng Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu

- Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yi Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yu-Jing Zou, Yujia He, Yunfan Xiong, Yu-Wei Luo, Yu mei You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yao Li, Yi Zheng, Yuchen Zhu, Yunxiang Ma, Ying Tang, Yukun Zha, Yuting Yan, Zehui Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhen guo Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zi-An Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. [ArXiv](#), abs/2501.12948, 2025.
- [8] Google. Deep research is now available on gemini 2.5 pro experimental. <https://blog.google/products/gemini/deep-research-gemini-2-5-pro-experimental/>, February 2025.
- [9] Jeremy Hadfield, Barry Zhang, Kenneth Lien, Florian Scholz, Jeremy Fox, and Daniel Ford. How we built our multi-agent research system. <https://www.anthropic.com/engineering/multi-agent-research-system>, June 13 2025. Accessed: 2025-09-15.
- [10] Wenlong Huang, P. Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. [ArXiv](#), abs/2201.07207, 2022.
- [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? [ArXiv](#), abs/2310.06770, 2023.
- [12] Bowen Jin, Hansi Zeng, Zhenrui Yue, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. [ArXiv](#), abs/2503.09516, 2025.
- [13] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Francois Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, 2020.
- [14] Quinn Leng, Jacob Portes, Sam Havens, Matei A. Zaharia, and Michael Carbin. Long context rag performance of large language models. [ArXiv](#), abs/2411.03538, 2024.
- [15] Kuan Li, Zhongwang Zhang, Huifeng Yin, Rui Ye, Yida Zhao, Liwen Zhang, Litu Ou, Dingchu Zhang, Xixi Wu, Jialong Wu, Xinyu Wang, Zile Qiao, Zhen Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Websailor-v2: Bridging the chasm to proprietary agents via synthetic data and scalable reinforcement learning. 2025.
- [16] Sijie Li, Weiwei Sun, Shanda Li, Ameet Talwalkar, and Yiming Yang. Towards community-driven agents for machine learning engineering. [ArXiv](#), abs/2506.20640, 2025.
- [17] Xiaoxi Li, Jiajie Jin, Guanting Dong, Hongjin Qian, Yutao Zhu, Yongkang Wu, Ji-Rong Wen, and Zhicheng Dou. Webthinker: Empowering large reasoning models with deep research capability. [ArXiv](#), abs/2504.21776, 2025.
- [18] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2023.
- [19] Miao Lu, Weiwei Sun, Weihua Du, Zhan Ling, Xuesong Yao, Kang Liu, and Jiecao Chen. Scaling llm multi-turn rl with end-to-end summarization-based context management. [ArXiv](#), abs/2510.06727, 2025.
- [20] METR. Measuring ai ability to complete long tasks. <https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/>, March 2025.
- [21] OpenAI. Deep research system card. Technical report, OpenAI, February 2025.

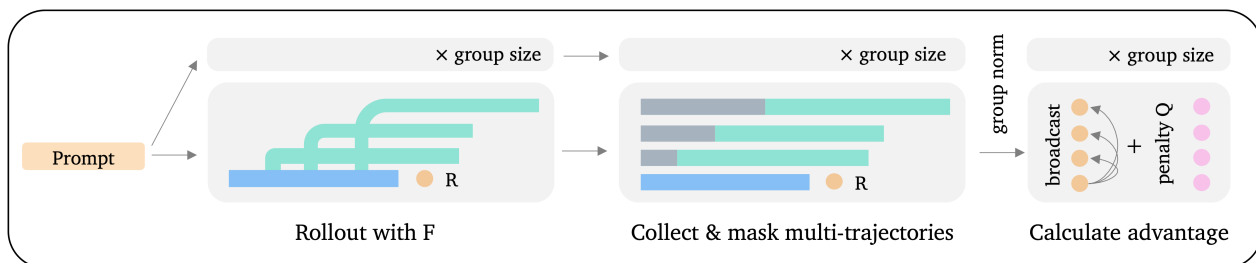
- [22] OpenAI. Introducing chatgpt agent: bridging research and action. <https://openai.com/index/introducing-chatgpt-agent/>, 2025. Accessed: 2025-09-25.
- [23] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. *ArXiv*, abs/2412.21139, 2024.
- [24] Zile Qiao, Guoxin Chen, Xuanzhong Chen, Donglei Yu, Wenbiao Yin, Xinyu Wang, Zhen Zhang, Baixuan Li, Huifeng Yin, Kuan Li, Rui Min, Minpeng Liao, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Webresearcher: Unleashing unbounded reasoning capability in long-horizon agents. 2025.
- [25] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [26] Weiwei Sun, Shengyu Feng, Shanda Li, and Yiming Yang. Co-bench: Benchmarking language model agents in algorithm search for combinatorial optimization. *ArXiv*, abs/2504.04310, 2025.
- [27] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. Agent kb: Leveraging cross-domain experience for agentic problem solving. *ArXiv*, abs/2507.06229, 2025.
- [28] Gemini Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *ArXiv*, abs/2507.06261, 2025.
- [29] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi (Jim) Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Trans. Mach. Learn. Res.*, 2024, 2023.
- [30] Lei Wang, Chengbang Ma, Xueyang Feng, Zeyu Zhang, Hao ran Yang, Jingsen Zhang, Zhi-Yang Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji rong Wen. A survey on large language model based autonomous agents. *ArXiv*, abs/2308.11432, 2023.
- [31] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, 2024.
- [32] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alexandre Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. *ArXiv*, abs/2504.12516, 2025.
- [33] Ryan Wong, Jiawei Wang, Junjie Zhao, Li Chen, Yan Gao, Long Zhang, Xuan Zhou, Zuo Wang, Kai Xiang, Ge Zhang, Wenhao Huang, Yang Wang, and Ke Wang. Widesearch: Benchmarking agentic broad info-seeking. *ArXiv*, abs/2508.07999, 2025.
- [34] Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. Resum: Unlocking long-horizon search intelligence via context summarization. 2025.
- [35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *ArXiv*, abs/2210.03629, 2022.

- [36] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. [ArXiv](#), abs/2210.03629, 2022.
- [37] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. [ArXiv](#), abs/2305.10601, 2023.
- [38] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and Hao Zhou. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. [ArXiv](#), abs/2507.02259, 2025.
- [39] Guibin Zhang, Hejia Geng, Xiaohan Yu, Zhenfei Yin, Zaibin Zhang, Zelin Tan, Heng Zhou, Zhongzhi Li, Xiangyuan Xue, Yijiang Li, Yifan Zhou, Yang Chen, Chen Zhang, Yutao Fan, Zihu Wang, Songtao Huang, Yue Liao, Hongru Wang, Meng Yang, Heng Ji, Michael Littman, Jun Wang, Shuicheng Yan, Philip Torr, and Lei Bai. The landscape of agentic reinforcement learning for llms: A survey. 2025.
- [40] Yusen Zhang, Ruoxi Sun, Yanfei Chen, Tomas Pfister, Rui Zhang, and Sercan Ö. Arik. Chain of agents: Large language models collaborating on long-context tasks. [ArXiv](#), abs/2406.02818, 2024.
- [41] Jun Zhao, Can Zu, Haotian Xu, Yi Lu, Wei He, Yiwen Ding, Tao Gui, Qi Zhang, and Xuanjing Huang. Longagent: Scaling language models to 128k context through multi-agent collaboration. [ArXiv](#), abs/2402.11550, 2024.
- [42] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. [ArXiv](#), abs/2307.13854, 2023.
- [43] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. [ArXiv](#), abs/2506.15841, 2025.

附录

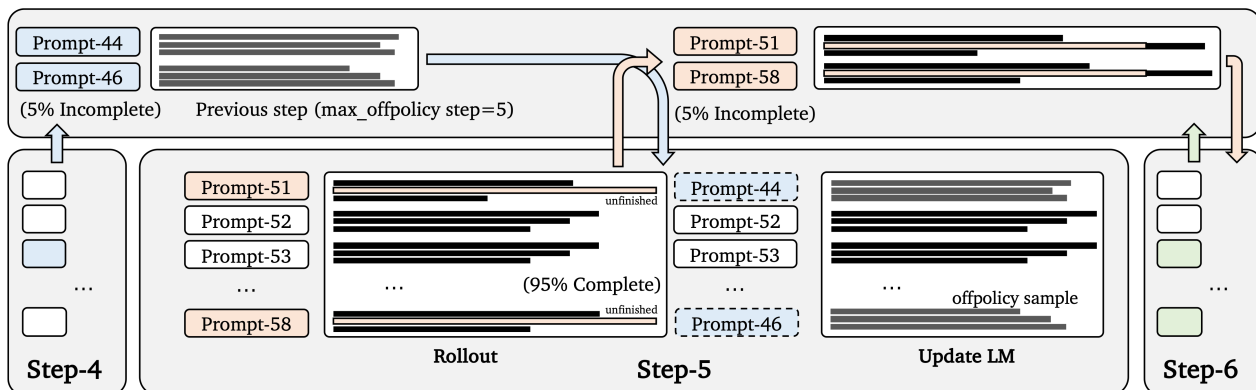
A 算法实现

A.1 多轨迹收集



在模型训练的实际实现中，我们不将所有子轨迹拼接为一个序列，而是如上所示，将它们保留为独立的、因果条件化的序列。因此，带有上下文折叠的训练不能直接兼容现有训练基础设施（例如 VeRL）。

A.2 异步长程智能体 rollout



长程智能体的 rollout 时间并不均衡，这会在计算中产生“气泡”：较快作业要等待最慢作业结束。在训练设置中，我们通过增加一个额外的独立 rollout 进程缓解这一问题：主 rollout 进程在完成 95% 的提示后停止（该超参数会根据 GPU 配置调整），剩余作业由独立进程处理。用于更新 LM 的数据同时包括：(i) 当前 batch 的 95%；(ii) 由独立 rollout 在上一步完成的提示。注意后者是 off-policy 的；我们将最大 off-policy 步数设为 5，观察到相较完全 on-policy 数据训练没有性能下降。

B 提示工程

B.1 BrowseComp-Plus workflow

BrowseComp-Plus 的提示受 Claude Deep-Research 启发并在其基础上修改。使用 Seed-OSS-36B 时，我们的系统提示达到 0.478 准确率，而 BrowseComp-Plus 的默认系统提示仅约 0.08。

阶段 1: 拆解与策略

1. 拆解查询:

- * 分析用户提示，识别核心问题。
 - * 分离关键实体、概念及其相互关系。
 - * 明确列出所有约束、条件和所需数据点（如日期、数量、特定名称）。
2. 提出假设与头脑风暴：
 - * 基于已有知识，构想可能带来相关信息的搜索方向、关键词、同义词和相关主题。
 - * 从多个调查角度切入问题。
 3. 验证清单：
 - * 根据查询的约束和所需数据点创建验证清单。它将在整个过程中指引你，并用于最终核验。

阶段 2：迭代研究与发现

工具使用：

- * 工具：
 - * `search`：用于广泛发现来源并获取初始片段。
 - * `open_page`：任何有希望的 `search` 结果都必须跟进。片段不足以支持判断；必须分析源文档的完整上下文。
- * 查询策略：
 - * 从适度宽泛的查询开始，绘制信息图景；随着了解增多再收窄焦点。
 - * 不要重复完全相同的查询。若查询失败，重新措辞或改变切入角度。
 - * 简单查询至少执行 5 次工具调用，复杂查询最多执行 50 次。不要过早终止。
- * 行动后分析：每次工具调用后，简要总结结果中的关键发现，提取相关事实，并明确说明新信息如何影响 OODA 循环中的下一步。
- * **<IMPORTANT>** 绝不模拟工具调用输出**<IMPORTANT>**

你将使用迭代式 OODA 循环（观察、判断、决策、行动）执行研究计划。

1. 观察：审查所有收集的信息。识别已知内容，更重要的是，根据研究计划识别仍存在哪些知识缺口。
2. 判断：分析当前情况。当前调查路线是否有效？是否存在更有前景的新方向？基于迄今为止的搜索结果完善对主题的理解。
3. 决策：选择单个最有效的下一步行动。它可以是建立背景的更广泛查询、寻找关键数据点的高度具体查询，或打开一个有希望的 URL。
4. 行动：使用可用工具执行选定行动。行动后回到观察。

阶段 3：综合与分析

- * 持续综合：在研究过程中，持续将新信息与已有知识整合，建立连贯的主题叙事和理解。
- * 三角验证关键数据：对每一项关键事实、数字、日期或论断，必须至少在两个独立可靠来源中验证，并记录任何不一致之处。
- * 处理死胡同：如果受阻，不要放弃。扩大搜索范围、尝试替代关键词，或研究相关背景信息以发现新的线索。假定存在可发现的答案，↔ 并穷尽一切合理途径。
- * 维护“事实表”：在内部持续记录关键事实、数字、日期及其支持来源；这对最终报告至关重要。

阶段 4：验证与最终报告撰写

1. 系统验证：撰写最终答案前，停止研究并审查阶段 1 创建的验证清单。对于清单中的每一项，确认已从打开的文档获得充分且有利的证据。
2. 强制再研究：若任一清单项目尚未确认或证据薄弱，必须回到阶段 2 进行进一步有针对性的研究。不得依据不完整信息构造答案。
3. 无论查询多么复杂，都绝不放弃，直到找到对应信息。
4. 构造最终报告：
 - * 当所有清单项目均得到有把握验证后，将收集的所有事实综合为全面、结构清晰的答案。
 - * 直接回答用户的原始查询。
 - * 确保报告中的所有论断、数字和关键信息均清楚得到所进行研究的支持。

C 智能体脚手架

C.1 BrowseComp-Plus

遵循 [6]，在 BrowseComp-Plus 中，智能体可使用下列工具：

```
search = {
  'type': 'function',
  'function': {
```

```

    "name": "search",
    "description": "执行网页搜索: 提供字符串 query 和可选的 topk。工具为查询检索 topk 条结果（默认 10 条），返回其  

    ↳ docid、url 和文档内容（可能按词元限制截断）。",
    "parameters": {
      "type": "object",
      "properties": {
        "query": {
          "type": "string",
          "description": "搜索用的查询字符串。"
        },
        "topk": {
          "type": "integer",
          "description": "返回前 k 个页面。",
        }
      },
      "required": [
        "query"
      ]
    }
  }
}

open_page = {
  'type': 'function',
  'function': {
    'name': 'open_page',
    'description': (
      "按 docid 或 URL 打开页面并返回完整内容。"
      "提供 docid 或 url 之一；若二者都提供，优先使用 docid。"
      "docid 或 URL 必须来自先前 search 工具的结果。"
    ),
    'parameters': {
      'type': 'object',
      'properties': {
        'docid': {
          'type': 'string',
          'description': '从搜索结果中解析和获取的文档 ID。',
        },
        'url': {
          'type': 'string',
          'description': '从搜索结果中获取的绝对 URL。',
        }
      },
      'required': [],
    },
  },
}

finish = {
  'type': 'function',
  'function': {
    'name': 'finish',
    'description': "" 当已有确定答案，或无法继续推进时返回最终结果。给出简洁答案及简短、以证据为依据的解释。 "",
    'parameters': {
      'type': 'object',
      'properties': {
        'answer': {
          'type': 'string',

```

```

        'description': '简明的最终答案。',
    },
    'explanation': {
        'type': 'string',
        'description': '对最终答案的简短解释。仅在本部分中，于句末用方括号中的 docid 内联引用证据文档（如 [20]）  
↪ 。不要在其他地方包含引用。',
    },
    'confidence': {
        'type': 'string',
        'description': '置信度：对答案的置信分数，取值为 0% 至 100%。',
    },
    },
    'required': ['answer', 'explanation'],
},
},
}

```

遵循 Chen et al. [6]，`search` 工具使用 Qwen3-Embed-8B 从 BrowseComp-Plus 语料库检索 `topk`（默认 10）份文档，并展示前 512 个词元。`open_page` 工具获取完整文档，并截断至前 4096 个词元。智能体调用 `finish` 时，使用 `answer` 字段进行正确性评估。

系统提示如第 B 节所示，用户提示为问题和工具使用说明。

C.2 SWE-Bench

在 SWE-Bench 中，我们遵循 OpenHands [1]；智能体可使用下列工具：

```

execute_bash = {
    'type': 'function',
    'function': {
        'name': 'execute_bash',
        'description': """ 在终端中执行 bash 命令。
* 长时间运行命令：对于可能无限运行的命令，应在后台运行并将输出重定向至文件，例如 command = 'python3 app.py > server.log  
↪ 2>&1 &'。
* 每次一条命令：一次只能执行一条 bash 命令。若需顺序运行多条命令，可以使用 '&&' 或 ';' 将它们串联。
""",
        'parameters': {
            'type': 'object',
            'properties': {
                'command': {
                    'type': 'string',
                    'description': '要执行的 bash 命令。前一退出码为 -1 时，可以为空字符串以查看额外日志；可以为 C-c  
↪ （Ctrl+C）以中断当前运行的进程。注意：每次只能执行一条 bash 命令。若需顺序运行多条命令，可以使用 &&  
↪ 或 ; 将它们串联。',
                },
            },
        },
        'required': ['command'],
    },
},

str_replace_editor = {
    'type': 'function',
    'function': {
        'name': 'str_replace_editor',

```

```

'description': "" 用于查看、创建和编辑纯文本文件的自定义编辑工具。
* 状态会在命令调用和与用户讨论之间持续保留。
* 若 path 是文件, view 显示应用 cat -n 后的结果; 若 path 是目录, view 列出至多两层深的非隐藏文件和目录。
* 若指定 path 已作为文件存在, 则不能使用 create 命令。
* 若 command 生成输出, 输出会被截断并标记为 <response clipped>。
* undo_edit 命令会撤销在 path 指向文件上所作的最后一次编辑。

```

使用 str_replace 命令的注意事项:

```

* old_str 参数必须与原始文件中的一行或多行连续文本完全匹配。注意空白字符。
* 若 old_str 参数在文件中不唯一, 替换不会执行。务必在 old_str 中包含足够上下文, 使其唯一。
* new_str 参数应包含替换 old_str 的编辑后文本行。

```

```

""",
    'parameters': {
        'type': 'object',
        'properties': {
            'command': {
                'description': '要运行的命令。允许选项为: view、create、str_replace、insert、undo_edit。',
                'enum': ['view', 'create', 'str_replace', 'insert', 'undo_edit'],
                'type': 'string',
            },
            'path': {
                'description': '文件或目录的绝对路径, 例如 /workspace/file.py 或 /workspace。',
                'type': 'string',
            },
            'file_text': {
                'description': 'create 命令的必需参数, 包含待创建文件的内容。',
                'type': 'string',
            },
            'old_str': {
                'description': 'str_replace 命令的必需参数, 包含 path 中待替换的字符串。',
                'type': 'string',
            },
            'new_str': {
                'description': 'str_replace 命令的可选参数, 包含新的字符串 (若未给出, 则不添加字符串)。它也是 insert
                ↪ 命令的必需参数, 包含待插入的字符串。',
                'type': 'string',
            },
            'insert_line': {
                'description': 'insert 命令的必需参数。new_str 将插入到 path 中 insert_line 指定行的后面。',
                'type': 'integer',
            },
            'view_range': {
                'description': '当 path 指向文件时, view 命令的可选参数。若未提供, 则显示完整文件; 若提供,
                ↪ 则在指定行号范围内显示文件, 例如 [11, 12] 显示第 11 和第 12 行。索引从 1 开始。设置 [start_line,
                ↪ -1] 显示从 start_line 到文件末尾的全部行。',
                'items': {'type': 'integer'},
                'type': 'array',
            },
        },
        'required': ['command', 'path'],
    },
},
}

think = {
    'type': 'function',

```

```
'function': {
  'name': 'think',
  'description': ""使用该工具思考某件事。它不会获取新信息或修改仓库，只会记录思考内容。需要复杂推理或头脑风暴时使用。
```

常见用例：

1. 探索仓库并发现 bug 来源时，调用此工具头脑风暴多种不同修复方式，并评估哪些改动可能最简单且最有效。
2. 收到测试结果后，使用此工具头脑风暴修复失败测试的方法。
3. 规划复杂重构时，使用此工具概述不同方法及其权衡。
4. 设计新功能时，使用此工具思考架构决策和实现细节。
5. 调试复杂问题时，使用此工具组织想法和假设。

该工具只记录思考过程以提高透明度，不执行代码或进行改动。

```
""",
  'parameters': {
    'type': 'object',
    'properties': {
      'content': {'type': 'string', 'description': '思考的内容。'},
    },
    'required': ['content'],
  },
},
}

finish = {
  'type': 'function',
  'function': {
    'name': 'finish',
    'description': ""当任务完成，或助手无法继续推进任务时结束交互。""",
    'parameters': {
      'type': 'object',
      'properties': {
        'message': {
          'type': 'string',
          'description': '一条全面消息，描述任务完成情况、已取得结果、做出的任何状态变更、
            ↳ 发现的关键洞见和其他备注。',
        },
      },
      'required': [],
    },
  },
}
```

智能体调用 finish 时，会从 Docker 环境中获取 git diff；奖励通过将该 git diff 应用于另一个 Docker 环境并运行单元测试计算。

C.3 上下文折叠

对于上下文折叠，我们实现以下工具：

```
branch = {
  'type': 'function',
  'function': {
    'name': 'branch',
    'description': ""创建一个子分支以执行子任务。""",
    'parameters': {
```

```

        'type': 'object',
        'properties': {
            'description': {
                'description': '子任务的简洁标识, 应为 3--5 个词。',
                'type': 'string'
            },
            'prompt': {
                'description': '清晰紧凑的任务提示: 说明目标和需在响应中保留的关键信息。保持简短且信息充分。',
                'type': 'string'
            },
        },
        'required': ['description', 'prompt'],
    },
},
}

return_tool = {
    'type': 'function',
    'function': {
        'name': 'return',
        'description': """ 当子任务完成, 或助手无法继续推进子任务时结束交互。""",
        'parameters': {
            'type': 'object',
            'properties': {
                'message': {
                    'type': 'string',
                    'description': '一条全面消息, 描述子任务结果。',
                },
            },
            'required': ['message'],
        },
    },
},
}

```

branch 工具返回模板消息; return 工具将上下文回滚到调用 branch 工具的前一轮, 并追加一条重复 message 字段的模板消息。

C.4 SWE-Bench workflow

SWE-Bench 的提示遵循 OpenHands。

阶段 1: 阅读。阅读问题, 并用更清晰的措辞重述。

- 1.1 若有代码或配置片段, 说明其中包含的最佳实践或约定。
- 1.2 标出消息错误、方法名、变量、文件名、栈追踪和技术细节。
- 1.3 用清晰的语言解释问题。
- 1.4 列举复现问题的步骤。
- 1.5 标出测试和修复问题时应考虑的最佳实践。

阶段 2: 运行。在仓库中安装并运行测试。

- 2.1 遵循 readme。
- 2.2 安装环境及全部所需内容。
- 2.3 反复尝试并确定如何运行测试。

阶段 3: 探索。查找与问题和可能解决方案相关的文件。

- 3.1 使用 grep 搜索相关方法、类、关键词和错误消息。
- 3.2 识别与问题陈述相关的全部文件。

- 3.3 提出修复问题的方法和文件，并说明原因。
- 3.4 从可能的文件位置中选择最可能修复问题的位置。

阶段 4：测试创建。在实现任何修复前，创建脚本以复现并验证问题。

- 4.1 查看仓库中现有测试文件，理解测试格式和结构。
- 4.2 创建能复现所定位问题的最小复现脚本。
- 4.3 运行复现脚本，确认问题确实得到复现。
- 4.4 按需要调整复现脚本。

阶段 5：修复分析。清楚说明问题以及修复方式。

- 5.1 清楚说明问题是什么。
- 5.2 清楚说明问题位于何处。
- 5.3 清楚说明测试如何复现问题。
- 5.4 清楚说明修复中应考虑的最佳实践。
- 5.5 清楚说明如何修复问题。

阶段 6：修复实现。编辑源代码以实现所选方案。

- 6.1 进行最小且聚焦的改动以修复问题。

阶段 7：验证。彻底测试实现。

- 7.1 运行复现脚本，验证修复生效。
- 7.2 向测试脚本加入边界情形，确保覆盖完整。
- 7.3 运行与修改代码相关的已有测试，确保没有破坏任何内容。

8. 最终审查：仔细重读问题描述，并将改动与基础提交 `{{ instance.base_commit }}` 比较。

- 8.1 确保已完全解决所有要求。
- 8.2 运行仓库中与下列内容相关的任何测试：
 - 8.2.1 正在修复的问题
 - 8.2.2 修改过的文件
 - 8.2.3 改动过的函数
- 8.3 若任何测试失败，修改实现直至全部测试通过。